

Normalizace databáze

Co je normalizace databáze

- **normalizace databáze** = postup návrhu relační databáze
- cílem je správně rozdělit data do tabulek
- omezuje zbytečné opakování dat
- chrání databázi před chybami při práci s daty
- používá se hlavně u relačních databází
- normalizace vede k lepší konzistenci dat

Normalizace pomáhá řešit:

- duplicitu dat
- špatně navržené tabulky
- problémy při vkládání dat
- problémy při úpravě dat
- problémy při mazání dat

Proč normalizovat

- data se neopakují zbytečně vícekrát
- jedna informace je uložena ideálně na jednom místě
- snižuje se riziko nekonzistence
- databáze se lépe udržuje
- vztahy mezi daty jsou jasnější
- změny se provádí bezpečněji

Příklad:

- název produktu nechceme ukládat u každé objednávky znovu
- produkt má být v tabulce `Product`
- objednávka na produkt pouze odkazuje pomocí `product_id`

Datové anomálie

Co jsou anomálie

- **anomálie** = problémy vznikající při špatném návrhu databáze
- typicky vznikají kvůli duplicitám
- normalizace se snaží těmto problémům zabránit

Update anomálie

- jedna informace je uložena na více místech
- při změně ji musíme upravit všude
- pokud na jedno místo zapomeneme, data si odporují

Příklad:

objednavka_id	zakaznik	adresa
1	Jan Novák	Praha
2	Jan Novák	Praha
3	Jan Novák	Brno

Problém:

- stejný zákazník má najednou dvě různé adresy
- není jasné, která je správná

Insert anomálie

- nejde vložit jednu informaci bez jiné
- databáze nás nutí uložit i data, která zatím nemáme

Příklad:

- chceme uložit nového zákazníka
- ale tabulka je navržena tak, že zákazník existuje jen v objednávce
- zákazníka bez objednávky neumíme uložit

Delete anomálie

- smazáním jednoho záznamu omylem smažeme i jinou důležitou informaci

Příklad:

- zákazník má jen jednu objednávku
- smažeme jeho poslední objednávku
- tím ztratíme i informaci o zákazníkovi

Normalizace vs. optimalizace

Normalizace

- rozděluje data do více tabulek
- omezuje duplicity
- zvyšuje konzistenci dat

- zlepšuje údržbu databáze
- často vyžaduje více vazeb a JOINů

Nevýhoda:

- složitější dotazy mohou být pomalejší
- často je potřeba spojovat více tabulek

Optimalizace

- cílem je zvýšit výkon databáze
- někdy se kvůli rychlosti data záměrně spojí zpět dohromady
- tomu se říká **denormalizace**
- používá se hlavně tam, kde je důležité rychlé čtení

Výhoda:

- méně JOINů
- rychlejší čtení dat

Nevýhoda:

- více duplicit
- větší riziko nekonzistence
- změny dat se musí hlídat opatrněji

V praxi

- produkční databáze bývají často normalizované
- analytické databáze bývají často denormalizované
- reálný návrh je kompromis mezi:
 - správností
 - výkonem
 - jednoduchostí
 - údržbou

OLTP a OLAP

OLTP

- **OLTP** = Online Transaction Processing
- databáze pro běžný provoz aplikace
- používá se v produkčních systémech
- typicky mnoho malých operací
- časté vkládání, úpravy a mazání dat

Příklady:

- e-shop
- bankovní systém
- rezervační systém
- školní evidence
- skladový systém

Vlastnosti:

- důraz na konzistenci
- důraz na transakce
- časté zápisy
- normalizovanější struktura
- data jsou rozdělena do více tabulek

Nevýhoda:

- složité čtení může vyžadovat více JOINů

OLAP

- **OLAP** = Online Analytical Processing
- databáze pro analýzu a reporting
- používá se pro vyhodnocování dat
- typicky méně zápisů
- hodně čtení a agregací

Příklady:

- statistiky prodeje
- reporting
- business intelligence
- datové sklady
- analýza chování uživatelů

Vlastnosti:

- důraz na rychlé čtení
- velké objemy dat
- časté agregace
- často denormalizovaná data
- méně JOINů

OLTP vs. OLAP

Vlastnost	OLTP	OLAP
Použití	běžný provoz aplikace	analytika a reporting
Operace	hodně malých CRUD operací	velké dotazy a agregace
Zápisy	časté	méně časté
Čtení	konkrétní záznamy	velké objemy dat
Struktura	spíše normalizovaná	často denormalizovaná
Důraz	konzistence, transakce	rychlé čtení, analýza

- Typický tok dat:

OLTP databáze → transformace dat → OLAP databáze / datový sklad

Základní pojmy

Primární klíč

- **primární klíč** = jednoznačný identifikátor řádku
- označuje se **PK**
- nesmí být `NULL`
- nesmí se opakovat

Příklad:

```
Student(student_id, first_name, last_name)
```

- `student_id` je primární klíč

Kandidátní klíč

- sloupec nebo skupina sloupců, která umí jednoznačně určit řádek
- kandidátní klíč by mohl být zvolen jako primární klíč

Příklad:

- `student_id`
- e-mail
- rodné číslo

Složený klíč

- klíč složený z více sloupců
- používá se často u vazebních tabulek

Příklad:

```
OrderItem(order_id, product_id)
```

- klíč tvoří kombinace `order_id` a `product_id`

Neklíčový atribut

- atribut, který není součástí žádného kandidátního klíče
- běžný datový sloupec

Příklad:

```
Student(student_id, first_name, last_name)
```

- `student_id` je klíč
- `first_name` a `last_name` jsou neklíčové atributy

Funkční závislost

- zápis: $X \rightarrow Y$
- znamená: hodnota `X` určuje hodnotu `Y`

Příklad:

```
student_id → first_name
```

Význam:

- podle `student_id` zjistíme jméno studenta
- jeden konkrétní `student_id` odpovídá jednomu konkrétnímu jménu

Normální formy

Co jsou normální formy

- **normální forma** = pravidlo pro správný návrh tabulky
- každá další forma zpřísňuje návrh
- cílem je odstranit duplicity a závislosti, které způsobují anomálie

Typicky se řeší:

- 0NF
- 1NF
- 2NF

- 3NF
- BCNF

0NF

Co je 0NF

- **0NF** = nenormalizovaná tabulka
- není to oficiální normální forma
- označuje stav před normalizací
- data jsou často „naházená“ v jedné tabulce

Typické problémy:

- více hodnot v jednom sloupci
- seznamy v buňce
- opakující se skupiny
- míchání více entit v jedné tabulce
- velké množství duplicit

Příklad 0NF:

order_id	customer_name	products	quantities
1	Jan Novák	Notebook, Myš	1, 2
2	Eva Svobodová	Klávesnice, Monitor	1, 1

Problém:

- ve sloupci `products` je více hodnot
- ve sloupci `quantities` je více hodnot
- špatně se s tím pracuje v SQL
- nejde jednoduše filtrovat a spojovat data

1NF

Co je 1NF

- **1NF** = první normální forma
- tabulka je v 1NF, když má atomické hodnoty
- každý sloupec obsahuje jednu hodnotu
- v buňce nesmí být seznam hodnot
- tabulka nesmí obsahovat opakující se skupiny

Atomická hodnota

- hodnota, která se v daném kontextu dál nedělí
- v jedné buňce má být jedna informace

Špatně:

```
Telefon = "777111222, 777333444"
```

Správně:

```
PersonPhone(person_id, phone)
```

Porušení 1NF

person_id	name	phones
1	Jan Novák	777111222, 777333444

Problém:

- ve sloupci `phones` je více telefonů
- hodnota není atomická

Oprava do 1NF

Tabulka `Person` :

person_id	first_name	last_name
1	Jan	Novák

Tabulka `PersonPhone` :

person_id	phone
1	777111222
1	777333444

Výsledek:

- každý telefon je samostatný řádek
- hodnoty jsou atomické
- data se lépe vyhledávají

Poznámka ke jménu

- Name = "Jan Novák" může být problém, pokud potřebujeme pracovat zvlášť se jménem a příjmením
- potom je lepší rozdělit na:
 - FirstName
 - LastName

2NF

Co je 2NF

- **2NF** = druhá normální forma
- tabulka musí být v 1NF
- každý neklíčový atribut musí záviset na celém kandidátním klíči
- řeší hlavně tabulky se složeným klíčem
- odstraňuje částečné závislosti

Částečná závislost

- neklíčový atribut závisí jen na části složeného klíče
- ne na celém klíči

Příklad porušení 2NF:

```
OrderItem(order_id, product_id, product_name, quantity)
```

Klíč:

```
(order_id, product_id)
```

Závislosti:

```
(order_id, product_id) → quantity  
product_id → product_name
```

Problém:

- quantity závisí na celé kombinaci objednávky a produktu
- product_name závisí jen na product_id
- product_name tedy nezávisí na celém složeném klíči
- to porušuje 2NF

Oprava do 2NF

Tabulka Product :

product_id	product_name
1	Notebook
2	Myš

Tabulka OrderItem :

order_id	product_id	quantity
100	1	1
100	2	2

Výsledek:

- název produktu je uložen jen v tabulce Product
- položka objednávky obsahuje jen odkaz na produkt
- odstranila se částečná závislost

3NF

Co je 3NF

- **3NF** = třetí normální forma
- tabulka musí být ve 2NF
- neklíčové atributy nesmí záviset na jiných neklíčových attributech
- odstraňuje tranzitivní závislosti

Tranzitivní závislost

- primární klíč určuje atribut A
- atribut A určuje atribut B
- tím pádem primární klíč určuje B nepřímo přes A

Zápis:

```
PK → A
A → B
PK → B
```

Problém:

- B závisí na neklíčovém atributu A
- to do 3NF nepatří

Příklad porušení 3NF

```
Employee(employee_id, employee_name, dept_id, dept_name)
```

Závislosti:

```
employee_id → employee_name  
employee_id → dept_id  
dept_id → dept_name
```

Problém:

- dept_name závisí na dept_id
- dept_id není hlavní klíč tabulky Employee
- vzniká tranzitivní závislost

Oprava do 3NF

Tabulka Department :

dept_id	dept_name
1	IT
2	HR

Tabulka Employee :

employee_id	employee_name	dept_id
10	Jan Novák	1
11	Eva Svobodová	2

Výsledek:

- název oddělení je uložen pouze v tabulce Department
- zaměstnanec na oddělení jen odkazuje
- odstranila se tranzitivní závislost

BCNF

Co je BCNF

- **BCNF** = Boyce-Coddova normální forma
- je přísnější než 3NF

- řeší speciálnější případy, které 3NF ještě může připustit
- používá se hlavně u složitějších závislostí a více kandidátních klíčů

Definice BCNF

- pro každou netriviální funkční závislost $X \rightarrow Y$ musí platit, že X je superklíč

Superklíč

- množina atributů, která jednoznačně určuje řádek
- může obsahovat i zbytečné atributy navíc

Příklad:

- `student_id` je superklíč
- `student_id + first_name` je také superklíč
- ale není minimální

Kandidátní klíč vs. superklíč

- kandidátní klíč je minimální superklíč
- superklíč může obsahovat atributy navíc

Jednodušší vysvětlení BCNF

- každá závislost v tabulce má vycházet z klíče
- pokud nějaký neklíčový atribut určuje jiný atribut, je to problém
- BCNF chce, aby určující strana závislosti byla vždy superklíč

Příklad porušení BCNF

Tabulka:

```
Vyuka(student, predmet, ucitel)
```

Předpoklady:

- student může mít daný předmět jen u jednoho učitele
- každý učitel učí jen jeden předmět

Závislosti:

```
(student, predmet) → ucitel  
ucitel → predmet
```

Kandidátní klíč může být:

(student, predmet)

Problém:

- ucitel → predmet
- ucitel není superklíč celé tabulky
- tabulka tedy není v BCNF

Možná oprava BCNF

Tabulka UcitelPredmet :

ucitel	predmet
Novák	Databáze
Svoboda	Programování

Tabulka StudentUcitel :

student	ucitel
Jan	Novák
Eva	Svoboda

Výsledek:

- závislost ucitel → predmet je oddělena
- každá tabulka má jasnější význam
- odstranil se problém s BCNF

Shrnutí normálních forem

Forma	Hlavní pravidlo	Co řeší
0NF	nenormalizovaná data	seznamy, opakující se skupiny
1NF	atomické hodnoty	více hodnot v jedné buňce
2NF	závislost na celém klíči	částečné závislosti
3NF	bez tranzitivních závislostí	závislosti mezi neklíčovými atributy
BCNF	každá závislost vychází ze superklíče	složitější závislosti

Jednoduchý postup normalizace

1. Najít entity

- zákazník
- objednávka
- produkt
- zaměstnanec
- oddělení

2. Najít atributy

- zákazník má jméno, e-mail, adresu
- produkt má název a cenu
- objednávka má datum

3. Najít klíče

- `customer_id`
- `product_id`
- `order_id`

4. Odstranit seznamy a opakující se skupiny

- tím se dostáváme do 1NF

5. Odstranit částečné závislosti

- hlavně u složených klíčů
- tím se dostáváme do 2NF

6. Odstranit tranzitivní závislosti

- neklíčové atributy přesunout do samostatných tabulek
- tím se dostáváme do 3NF

7. Zkontrolovat BCNF

- každá funkční závislost má vycházet ze superklíče

Denormalizace

Co je denormalizace

- **denormalizace** = záměrné porušení normalizačních pravidel kvůli výkonu
- některá data se duplicitně uloží
- cílem je zrychlit čtení
- často se používá u OLAP databází

Příklad:

- místo častého JOINu s tabulkou `Product`
- uložíme `product_name` přímo do tabulky s reportem

Výhoda:

- rychlejší dotazy
- méně JOINů
- jednodušší čtení

Nevýhoda:

- duplicita dat
- riziko nekonzistence
- složitější aktualizace